

函数文件与最小测试实验

把不一致缩成一个能重现的问题

这次实验的主要结果是一份可定位的故障报告，以及由小测试支撑的模块化校准程序。数据和两种规则沿用001，数值答案不变。只使用Python标准库；Notebook需要JupyterLab和ipykernel，不需要GPU、额外数据下载或付费API。

先读讲义中函数边界、字典和路径三节，再运行实验。不要把下面步骤合并成一次“运行全部就好”：第一次学习应观察每个中间产物；最后才用重启并全部运行检验可重复性。

一 对照程序结构

experiment.py包含正式读取、验证、模型、评分与记录函数。faults.py保存四个故意写错的函数，它们不参与真实结果。test_experiment.py给出手算断言、输入边界和启动位置测试。experiment.ipynb按阅读顺序调用这些内容。data中的CSV与001相同；已公开的测试标签仅用于流程练习。

打开experiment.py，先找到read_csv_records、mean_absolute_error、PredictionRule、AffineRule、LookupRule和run。函数定义并不会立即执行函数体。到最后的主入口条件成立时，run才会组织一次完整实验。Notebook导入这些名称，随后自己按顺序调用。

在纸上写四份输入输出约定：读取函数接收路径及表头列表，返回非空记录列表；模型predict接收数值输入列表，返回同长度预测列表；标签对齐函数按唯一编号安排答案；MAE函数接收等长非空数值列表并返回标量。每份约定都加一个应该拒绝的例子。

二 看一条记录怎样改变类型

运行Notebook的读取单元。CSV里的x是文本，读取函数把它转换为浮点数，并保留id字符串。第一行应成为 {'id': 'T01', 'x': 1.0, 'y': 3.0}。核对 rows[0]["x"]打印1.0，type(...)打印float类型；type是查看对象类型的内置函数。

讲义展示的核心代码用for逐项处理，以便逐步跟踪。完整文件中还有少量检查工具：

isinstance(value, (int, float))检查值是否属于允许的两种类型，圆括号中的两个类型组成不可变的元组；None表示这里还没有数值；is None检查是否仍为这个标记。布尔值在Python中与整数有特殊关系，因此程序单独拒绝True和False，避免把“是/否”误当校准测量。

str(row_number)把行号转换成可拼接的文字，用来写清错误位置；raise ... from error在补充上下文时保留原始错误来源。这些辅助写法不改变预测公式。读到某个陌生工具时，先看它在边界检查中负责什么，不要把工具数量误当模型复杂度。

三 独立检查模型和评分

先不读测试标签。创建偏移量为1的AffineRule，对 [1.5, 2.5, 3.5, 4.5] 预测，结果应为 [4, 6, 8, 10]。创建偏移量为2的另一个实例，对3预测应为8；第一个实例对3仍为7。修改第二个实例不能改变第一个实例的偏移量。

用训练记录创建LookupRule。对训练输入3应输出8；对未出现的2.5应输出0。这两次调用分别检查命中和未命中路径。不能只测训练点就以为后备逻辑已经验证。

对评分函数使用独立纸笔答案：真实 [3, 5]、预测 [2, 6]，两次误差都为1，MAE为1；真实 [3, 5, 8, 9, 11]、预测 [3, 5, 7, 9, 11]，总误差1除以5为0.2。Notebook用很小容差比较浮点数。

四 分别运行完整脚本与测试

在本单元目录依次执行：

```
python experiment.py
python test_experiment.py
```

第一条命令在outputs保存冻结预测、逐条预测、汇总指标与版本。第二条运行测试，打印status为passed及九组测试内容；临时故障文件在独立临时目录内创建并自动清理，不改原始数据。不要用Python的优化选项关闭断言。

测试包括三条手算指标、四种无效指标输入、实例状态隔离与输入不变、基类接口与编号对齐、六种坏CSV、三种异常故障与错误除数、完整实验指标、独立导入，以及改变启动目录。这里的九组各自包含一项或多项检查，不把“九组”误读成只有九个断言。

测试使用标准库tempfile创建临时目录，subprocess启动全新Python进程，importlib在明确路径导入模块。这些是验证用辅助工具，本讲不要求从零重写它们；它们解决的是“旧进程状态会不会掩盖步骤”，不参与模型计算。输出只保留测试名称，不记录私人的实际目录。

五 亲手追踪三种异常

先调用 `faults.wrong_indices([1, 2, 3])`。合法位置是0、1、2，错误循环依次访问1、2、3，到3时触发IndexError。请指出它还漏掉了位置0。如果只是删去最后一次访问，仍然可能丢掉首项。正确做法可以直接遍历列表元素，或使用从0到长度减1的索引。

再调用 `faults.unconverted_prediction("1.5")`。字符串乘2得到重复文字，随后与整数1相加触发TypeError。修正应放在输入边界：把可解释的文本显式转成浮点，并拒绝坏文本。不要改成字符串拼接来让表达式“能运行”，那会把数值预测变成另一件事。

最后在没有train.csv的临时目录调用`wrong_working_directory`，得到FileNotFoundError。课程包里的数据位于data/train.csv，且脚本入口与Notebook入口的基准不同。修正是明确定位，不是把原始文件复制到每个可能目录，造成多个版本互相冲突。

Notebook捕捉这些预定错误并打印异常类型，避免让整本在故障示范处停止。自己探索调用时看到错误堆栈很正常，先读末行类型，再找故障函数位置。不应通过把所有异常都pass掉来“修复”。

六 抓住不报错的错误

wrong_denominator把总误差固定除以3。给它两条误差均为1的记录，它会返回约0.667，程序没有异常。独立手算要求平均为1，所以断言失败。再用三条都错1的记录，它碰巧得到1并通过；单一测试恰好通过不能证明实现正确。

写一个不同长度的测试是自然的下一步。可以用四条真实值均为0、预测均为2，正确MAE为2，错误程序却返回约2.667。这类测试让“平均必须除以实际条数”成为可检查的要求。

七 重启与换目录

在Notebook中完成全部步骤后，重启内核并从第一格运行。核对没有NameError，也没有只存在于旧输出里的成功记录。实验本将结果保存到notebook_outputs，与脚本outputs分开。

test_experiment.py还会从无关工作目录启动新进程，再调用脚本函数。这一步验证以脚本文件为基准找到数据，结果与原目录一致。另一个新进程只导入一个不带数据的脚本副本，确认导入不要求立即读数据，也不生成结果文件。

课程验证使用新进程中的真实IPython InProcessKernel顺序执行Notebook。受制作环境限制，没有检验浏览器Jupyter界面、外进程内核传输或每种系统的Anaconda安装；实际运行方式写在verification.json。不要把这条执行记录扩展成对所有平台的保证。

八 故障报告的最小内容

为三个异常各写一段：最小输入、调用方式、预期、实际异常类型、出错步骤、修正与回归测试。为错误除数另写一段，明确它“没有报错但数值错误”。最后保存两种规则原始实验的逐条结果和六个指标，确认重构没有改变任务。

实验完成的判断不是文件数量，而是别人能否按你的报告重新触发故障，再用同一组测试确认修正。练习完整推理见answers.pdf，建议完成自己的报告后再阅读。